

Zadání 2. projektu ATA LS 2024/25

Cílem projektu je praktické cvičení dynamické analýzy na základě sledování událostí. Cvičení by mělo pokrýt runtime verifikaci programů a sledování platností formulí za běhu.

Vytvořte dynamický analyzátor, který bude sledovat porušení vlastností daného programu na základě aktuálního záznamu událostí. Dynamický analyzátor bude také reportovat základní pokrytí využití vozíku.

Popis testovaného produktu

Testovacím subjektem je řídicí software vozíku v hypotetické továrně. Specifikace chování (následující 2 odstavce nezměněny od 1. projektu):

Řízení vozíku v robotické továrně

V robotické továrně je autonomní motorizovaný vozík pro převoz materiálu potřebného k výrobě. Vozík má předem určené trasy a více zastávek. Řídicí systém vozíku má na starosti plánování další zastávky a monitorování zátěže a využití kapacity vozíku. Vozík zpracovává požadavky o přesun materiálu. Materiál, resp. požadavek na přesun, je určený svojí zdrojovou a cílovou stanicí, váhou a binární prioritou (přesun je nebo není prioritní). Každý materiál má při převozu alokovan jeden slot vozíku. Pokud je materiál pro převoz větší než jeden slot nebo je příliš těžký, bude rozdělen do více slotů. O toto rozdělení se však kontrolní systém vozíku nestará. Uvažujte, že vozík je používán v ideálním prostředí (systémy s ním komunikující splňují veškeré předpoklady, data odpovídají reálným stavům). Vozík má pro převoz materiálu celkem 1 až 4 sloty. Vozíky jsou 3 druhů podle maximální nosnosti: 50 kg, 150 kg a 500 kg. Vozíky s nejmenší nosností mají nejméně 2 sloty, vozíky s největší nosností mají maximálně 2 sloty.

Specifikace chování

Řídicí systém automaticky generuje požadavky na přemístění materiálu mezi zastávkami v robotické továrně. Po přijetí požadavku vozík vyhodnotí trasu a zahájí nakládku materiálu. Pokud vozík do 1 minuty nenaloží materiál, řídicí systém nastaví materiálu prioritní vlastnost na hodnotu 1. Pokud takový materiál stále není naložen do další 1 minuty a vozík má volnou kapacitu, materiál se okamžitě naloží a vozík přepne do režimu pouze-vykládka. V tomto režimu vozík ignoruje jiné požadavky na převoz a zamíří přímo k cílové zastávce prioritního materiálu. Po vyložení veškerého prioritního materiálu přepne zpět do režimu normální. V tomto režimu optimalizuje trasu tak, aby minimalizoval celkovou přepravní vzdálenost a umožnil současnou nakládku a vykládku více materiálů v rámci své plánované cesty. Pořadí vyzvednutí materiálů nezávisí na pořadí vytvoření požadavků, ale na optimalizaci trasy a aktuálním zatížení vozíku. Pokud jsou všechny sloty vozíku obsazené nebo by naložením materiálu došlo k překročení maximální nosnosti, vozík materiál nenaloží a pokračuje k nejbližší cílové zastávce pro vyložení aktuálního nákladu. Po uvolnění kapacity se znovu vyhodnotí možnost naložení čekajícího materiálu.

Záznam událostí

Vozík s každou aktivitou vypisuje na standardní výstup svůj aktuální stav. Následují formáty hlášení (ostatní řádky hlášení zde nedefinované ignorujte):

Žádost o naložení (v čase TIME ze stanice SRC do stanice DST s nákladem CONTENT o váze W):

TIME requesting SRC DST CONTENT W

Počátek nakládání (v čase TIME s vozíkem ve stanici POS nákladu CONTENT o váze W do slotu SLOT):

TIME loading POS CONTENT W SLOT

Počátek vykládání (v čase TIME s vozíkem ve stanici POS nákladu CONTENT o váze W ze slotu SLOT):

TIME unloading POS CONTENT W SLOT

Pohyb vozíku (v čase TIME ze stanice POS1 do sousední stanice POS2):

TIME moving POS1 POS2

Kritérium pokrytí

Aby bylo možné vyhodnotit testovací sadu, implementujte vyhodnocení kritéria pokrytí zaměřené na obsazenost jednotlivých slotů vozíku a požadavků z jednotlivých stanic. Vámi implementovaný analyzátor na konci testování reportuje pokrytí. Kritérium pokrytí vozíku a požadavků je založeno na ADC (All-Defs Coverage), ve kterém proměnnou definujeme jako slot a jeho definici jako stanici naložení. Jinými slovy je kritérium pokrytí 100%, pokud ve všech existujících stanicích byl alespoň jednou naložen náklad do každého slotu vozíku. Pro 4 sloty vozíku a 4 existující stanice je to celkem 16 testovacích požadavků.

Monitorované vlastnosti

Dynamický analyzátor bude monitorovat platnost následujících vlastností (předpokládejte vozík se 4 sloty a nosností 150):

1. Vozík nesmí nakládat na obsazený slot.
2. Vozík nesmí vykládat z volného slotu.
3. Náklad se musí vyložit, pokud je vozík v cílové stanici daného požadavku.
4. (volitelné) Každý požadavek o přesun musí někdy způsobit nakládku.
5. Vozík nesmí nakládat ve stanici, pokud na to neexistoval požadavek.
6. Nesmí být naloženo více než 4 náklady.
7. Vozík ani žádný jeho slot nesmí být přetížen.
8. (volitelné) Každý naložený náklad se musí vyložit.
9. (volitelné) Vozík nesmí být nečinný, pokud existuje žádost o přesun.
 - Nečinnost vozíku znamená, že je ve stavu idle po nenulovou dobu. Jinými slovy, že neexistuje následující akce vozíku se stejným časem, ve kterém vozík přešel do stavu idle.

Úkoly

1. Vytvořte program v jazyku Python, který čtením standardního vstupu průběžně analyzuje správné chování vozíku.
2. Přeložte monitorované vlastnosti do monitorovacích automatů.
 - Omezený počet parametrů eliminujte rozvinutím automatu.
 - Neomezený počet parametrů ve vlastnostech řešte instanciací monitorů za běhu.
 - Automaty implementujte v souboru `cart_monitor.py`.
 - V případě porušení vlastnosti vypište na standardní výstup základní report: `TIME:error: popis porušené vlastnosti`. Vícenásobné porušení v daném čase reportovat nemusíte.
 - Grafovou reprezentaci automatů odevzdejte v souboru `automaty.pdf`.
3. Implementujte výpočet výše definovaného kritéria pokrytí. Po skončení analýzy vypište výsledné pokrytí (v procentech, zaokrouhleno dolů): `Coverage XX%`
4. Rozšiřte testovací vstupy v souboru `requests.csv` o náhodně vygenerované požadavky přesunu nákladu:
 - řádek obsahuje (čárkou oddělený seznam): čas příchodu požadavku, zdrojová stanice, cílová stanice, váha, popis nákladu

Odevzdání

Odevzdejte archiv `proj2.zip` obsahující následující soubory:

```
proj2.zip
+- automaty.pdf           - diagramy automatů, případné komentáře a popis
  |                       - implementace, reportování nalezených chyb
+- cart_monitor.py       - implementovaný dynamický analyzátor
`- requests.csv          - vámi vygenerované testovací vstupy
```

Poznámky a hodnocení

Pomocné soubory najdete v archivu (resp. adresáři) `cartmonitor.zip`. Vozík, jeho řízení a prostředí továrny je implementováno v adresáři `cartctl`. Testy a skripty pro tvorbu logů jsou v adresáři `tests`. Tvorba logů (`tests/*.txt`) lze provést pomocí příkazu `make` v kořeni zdrojových souborů:

```
cd cartmonitor
make [monitor]
```

V souboru `bad_log.txt` je příklad sekvence událostí, která v čase 12 porušuje první vlastnost (vozík nakládá do již naloženého slotu). Další logy můžete získat po spuštění `make monitor`.

K dobrému hodnocení přispěje:

- správná implementace všech nevolitelných vlastností,
- reportování chyby v případě porušení vlastnosti,
- výpočet pokrytí,
- shoda mezi monitorovacími automaty a jejich implementací v analyzátoru (např. pomocí komentářů v kódu s identifikátory stavů).

Primárním cílem projektu je vytvoření dynamického analyzátoru. Generování vstupních testovacích dat je nepovinné a slouží pouze jako pomůcka pro demonstraci analyzátoru nebo reprodukci chybového stavu. Plného počtu bodů je možné dosáhnout i bez implementace monitorů vlastností označené jako “volitelné”. Volitelné vlastnosti jsou parametrické (s neomezeným počtem hodnot) a jejich implementace může zvýšit výsledné hodnocení.

Příklady a spuštění

```
$ cd tests
$ cat requests.csv
10,A,B,20,helmet
12,A,B,40,braceletR
```

```
$ ./generate_log.sh requests.csv 2>/dev/null
10 requesting A B helmet 20
10 loading A helmet 20 0
12 requesting A B braceletR 40
12 loaded A helmet
12 idle A
12 loading A braceletR 40 1
14 loaded A braceletR
14 idle A
14 moving A B
34 moved to B
34 idle B
34 unloading B helmet 20 0
36 unloaded B helmet
36 idle B
36 unloading B braceletR 40 1
38 unloaded B braceletR
38 idle B
38 stop
...
```

```
$ ./generate_logs.sh requests.csv 2>/dev/null | ./cart_monitor.py
All properties hold.
CartCoverage 12%
```

```
$ ./cart_monitor.py <bad_log.txt
12:error: loading into an occupied slot #0
CartCoverage 6%
```